Data Pre-processing

Ir. Fandy Setyo Utomo, S.Kom., M.Cs., Ph.D

PSIFW074 – Pembelajaran Mesin

Program Studi Informatika - Universitas AMIKOM Purwokerto

Capaian Pembelajaran Praktikum:

Mahasiswa Mampu membangun model data pre-processing pada studi kasus pembelajaran mesin.

Software yang diperlukan dalam pembelajaran:

1. Python
2. pandas (https://pandas.pydata.org/)
3. Spyder (*Scientific Python Development Environment*)

Tahapan Praktikum:

1. Import seluruh pustaka yang dibutuhkan untuk operasi data pre-processing pada IDE Spyder:

```
#import libraries
import numpy as np
import pandas as pd
import sklearn
```

Catatan:

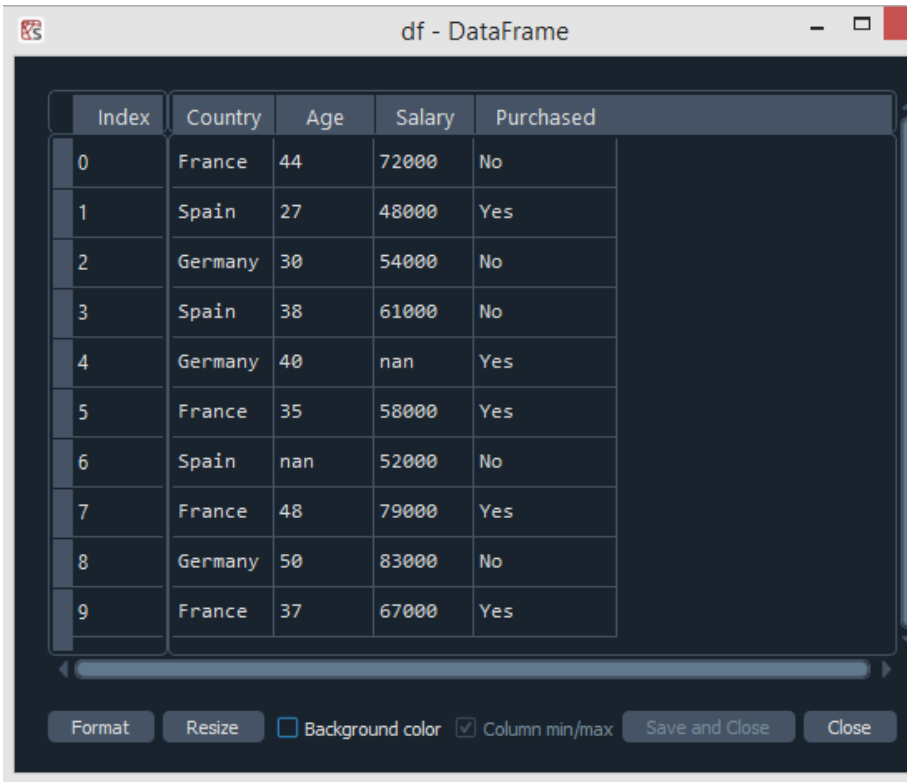
- *NumPy (Numerical Python)* merupakan pustaka yang berfungsi untuk komputasi numerik. Pustaka ini menyediakan struktur data array yang efisien dan operasi matematika untuk melakukan manipulasi data numerik, analisis statistik, dan perhitungan ilmiah. Informasi lebih detil tentang NumPy dapat diakses pada website berikut: https://numpy.org/
- *Pandas* merupakan pustaka yang berfungsi untuk analisis data dan manipulasi data tabular. Informasi lebih detil tentang pustaka ini dapat diakses pada website berikut: https://pandas.pydata.org/
- *Scikit-learn* atau *sklearn* adalah pustaka yang berfungsi menyediakan algoritma pembelajaran mesin (*machine learning*) dan alat-alat untuk mempermudah pengembangan model *machine learning*. Informasi lebih detil tentang pustaka ini dapat diakses pada website berikut: https://scikit-learn.org/stable/

2. Import dataset dengan menuliskan perintah berikut pada IDE Spyder:

```
#import dataset
dataset=pd.read_csv('Data.csv')
df=pd.DataFrame(dataset)
```

Catatan:

- Pastikan file Python telah disimpan dalam folder yang sama dengan file Data.csv
- Amati isi variable ***df*** melalui menu *variable explorer*:



Index	Country	Age	Salary	Purchased
0	France	44	72000	No
1	Spain	27	48000	Yes
2	Germany	30	54000	No
3	Spain	38	61000	No
4	Germany	40	nan	Yes
5	France	35	58000	Yes
6	Spain	nan	52000	No
7	France	48	79000	Yes
8	Germany	50	83000	No
9	France	37	67000	Yes

Gambar 1. Isi variabel ***df***

3. Ekstrak variabel independen dengan menuliskan perintah berikut pada IDE Spyder:

```
#Ekstract independent variables
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
```

Catatan:

- Penjelasan lebih detil terkait properti ***iloc*** untuk mengindeks dan mengekstrak variabel independen dapat diakses pada website: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.iloc.html
- **[:, :-1]** merupakan notasi *slicing* yang digunakan untuk memilih semua baris ("**:**") dan semua kolom, kecuali kolom terakhir ("**[:-1]**").

- **[:, -1]** merupakan notasi *slicing* yang digunakan untuk memilih semua baris ("**:**") pada *DataFrame*, dan hanya kolom terakhir ("**-1**").
- Amati isi variabel ***X*** melalui menu *variable explorer* dan amati perbedaannya dengan Gambar 1.

X - NumPy object array (read only)

	0	1	2
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	nan
5	France	35.0	58000.0
6	Spain	nan	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

Gambar 2. Isi variabel ***X***

- Amati isi variabel ***y*** melalui menu *variable explorer*!

4. Identifikasi *missing values* pada data dengan menuliskan perintah berikut pada IDE Spyder:

```
1. #identifikasi missing values
2. print(df.info())
3. mv=df.isna().sum()
4. print('\nJumlah missing value tiap kolom:\n', mv)
```

Catatan:

- **df.info()** digunakan untuk mengetahui atribut dan tipe data pada DataFrame.
- **.isna()** digunakan untuk mengecek berapa data yang bernilai *NaN* dan **.sum()** menjumlahkannya secara default untuk masing-masing kolom DataFrame.
- Amati output yang diberikan dari hasil eksekusi kode program tersebut pada jendela *IPython Console*. Berikut ini output yang diberikan untuk baris kode ke-4:

```
Jumlah missing value tiap kolom
 Country      0
Age           1
Salary        1
Purchased     0
dtype: int64
```

Gambar 3. Informasi Missing Value Tiap Kolom

Berdasarkan Gambar 3 dapat disimpulkan bahwa pada kolom ***Age*** terdapat 1 data yang bernilai *NaN* (*missing value*), demikian pula dengan kolom ***Salary***.

5. Terdapat beberapa cara/metode untuk menyelesaikan permasalahan *missing value*. Cara yang termudah dengan: (1) Izinkan *missing value*; (2) Hapus data yang memiliki *missing value*; atau (3) Isi *value NaN* (*missing value*) tersebut dengan *value* yang lain (biasanya interpolasi data, mean, median, atau teks tertentu).

Solusi 1: Hapus data yang memiliki *missing value*

Solusi ini dapat diimplementasikan dengan menuliskan perintah berikut pada IDE Spyder:

```
'''
Solusi 1: Hapus data yang memiliki missing value
'''
df1=df.copy()

#ukuran data awal
print("Sebelum: ", df1.shape)

#hapus baris yang memiliki missing value
df1.dropna(inplace=True)

#ukuran data akhir
print("Setelah: ", df1.shape)
```

Catatan:

- ***df.copy()*** digunakan untuk membuat salinan (*copy*) DataFrame. Dokumentasi lengkap dapat dipelajari pada website: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.copy.html
- ***df1.shape*** digunakan untuk memperoleh informasi ukuran dimensi DataFrame dalam bentuk *tuple*. Dokumentasi lengkap dapat dipelajari pada website: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.shape.html

- ***df1.dropna()*** digunakan untuk menghapus baris yang memiliki *missing value*. Parameter ***inplace=True*** artinya instruksi untuk memodifikasi DataFrame ***df1*** daripada membuat DataFrame baru. Dokumentasi lengkap dapat dipelajari pada website: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.dropna.html
- Amati isi variabel DataFrame ***df1***! Bandingkan dengan isi variabel pada DataFrame ***df***! Apa perbedaannya?

Solusi 2: Isi missing value dengan value lain

Solusi ini dapat diimplementasikan dengan menuliskan perintah berikut pada IDE Spyder:

```
'''
Solusi 2: Isi missing value dengan value lain
'''
df2 = df.copy()

#isi missing value dengan nilai kolom rata-rata (mean)
df2['Age'].fillna(df['Age'].mean(), inplace=True)
df2['Salary'].fillna(df['Salary'].mean(), inplace=True)
```

Catatan:

- Penjelasan lebih detil mengenai ***DataFrame.fillna*** dapat diakses melalui website: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html, sedangkan penjelasan mengenai ***DataFrame.mean*** dapat diakses melalui website: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.mean.html.
- Amati isi variabel DataFrame ***df2***! Bandingkan dengan isi variabel pada DataFrame ***df***! Apa perbedaannya?

Latihan:

Ubah nilai *NaN* (*missing value*) menjadi nilai Median!

Selain pergantian nilai *NaN* (*missing value*) dengan nilai *mean* di dalam DataFrame, pergantian nilai juga dapat dilakukan di dalam variabel ***X*** yang berisi data dari *independent variables*: *Country*, *Age*, *Salary*. Pergantian tersebut memanfaatkan pustaka *Scikit-Learn* yang diimplementasikan dengan menuliskan perintah berikut pada IDE Spyder:

```
'''
Isi missing value pada variabel X
'''
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.transform(X[:, 1:3])
```

Catatan:

- Class ***sklearn.impute.SimpleImputer*** menyediakan fitur imputer univariat untuk melengkapi *missing value* menggunakan statistik deskriptif (misalnya mean, median, atau *most frequent*) di setiap kolom, atau menggunakan nilai konstan. Dokumentasi lebih lengkap terkait class ini dapat diakses melalui website: https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html dan https://scikit-learn.org/stable/modules/impute.html.
- **X[:, 1:3]** dapat diartikan sebagai pengambilan semua baris dari array ***X***, tetapi hanya kolom dengan indeks 1 hingga 2 (kolom kedua dan ketiga).
- Amati isi variabel ***X***! Apa perbedaannya dengan isi variabel ***X*** sebelumnya?

Latihan:

Ubah nilai *NaN* (*missing value*) pada variabel ***X*** menjadi nilai Median atau nilai yang paling sering muncul (*most frequent*)!

PENGODEAN DATA KATEGORIKAL (*CATEGORICAL DATA*)

Model Pembelajaran Mesin berbasis pada persamaan matematika. Penyimpanan data kategorikal (selain numerik) dalam persamaan akan menyebabkan permasalahan karena hanya diperlukan angka di dalam persamaan. Di dalam studi kasus praktikum ini terdapat 2 variabel kategorikal, yaitu *Country* and *Purchased*. Oleh sebab itu, kedua variabel kategorikal tersebut perlu dikodekan.

6. Variabel *Country* dapat dikodekan menggunakan ***LabelEncoder*** dengan instruksi sebagai berikut:

```
#pengodean kolom Country dengan LabelEncoder
from sklearn.preprocessing import LabelEncoder

label_encoder_x= LabelEncoder()
X[:, 0]= label_encoder_x.fit_transform(X[:, 0])
```

Catatan:

- Class ***sklearn.preprocessing.LabelEncoder*** untuk pengodean label dengan nilai antara 0 dan n_classes-1. Informasi terkait class ini dapat diakses pada website: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html.
- **X[:, 0]** dapat diartikan sebagai pengambilan semua baris dari array ***X*** dan hanya kolom dengan indeks 0 (*Country*).
- Amati isi variabel ***X***. Apa perbedaan antara isi variabel tersebut saat ini dengan sebelumnya?

Selanjutnya, variabel *Purchased* dapat dikodekan dengan cara yang sama menggunakan ***LabelEncoder*** dengan instruksi sebagai berikut:

```
#pengodean kolom Purchased dengan LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
```

Catatan:

Amati isi variabel ***y***. Apa perbedaan antara isi variabel tersebut saat ini dengan sebelumnya?

PEMBAGIAN DATASET

Pembagian dataset merupakan langkah selanjutnya dalam pra-pemrosesan data. Setiap dataset untuk model pembelajaran mesin dibagi menjadi dua, yaitu set data latih (*training set*) dan set data uji (*test set*).

7. Instruksi berikut ini digunakan untuk membagi dataset menjadi training set dan test set:

```
#pembagian dataset menjadi training dan test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

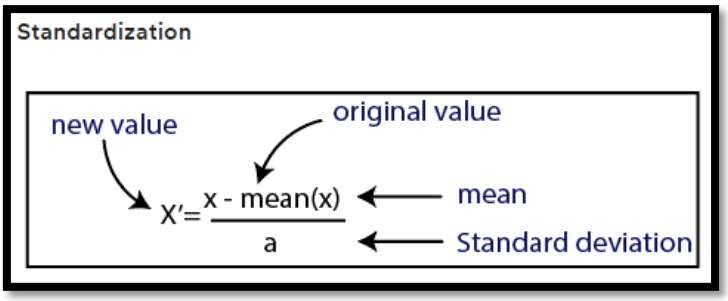
Catatan:

- Fungsi ***sklearn.model_selection.train_test_split*** digunakan untuk membagi array atau metrik menjadi subset pelatihan dan pengujian acak. Dokumentasi terkait fungsi ini dijelaskan lebih detil pada website berikut: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html#sklearn.model_selection.train_test_split. Penjelasan mengenai varian *Splitter Classes* dan *Splitter Functions* pada ***sklearn.model_selection*** dapat diakses pada: https://scikit-learn.org/stable/modules/classes.html#module-sklearn.model_selection.
- **test_size = 0.2** artinya ukuran set data uji = 20%, sedangkan set data latih berukuran 80%.
- Amati variabel ***X_train, X_test, y_train, y_test*** melalui *variable explorer*. Apa simpulan hasil pengamatan anda?

FEATURE SCALING

Feature scaling merupakan metode untuk menstandarisasi variabel independen suatu dataset dalam rentang tertentu. Dengan kata lain, *Feature scaling* membatasi rentang variabel. Di dalam studi kasus praktikum ini terdapat 2 variabel yang tidak memiliki skala yang sama yaitu *Age* and *Salary* (seperti yang ditunjukkan pada Gambar 1). Dalam skenario seperti ini, jika dikalkulasi dua nilai apapun dari kolom usia dan gaji, nilai gaji akan mendominasi nilai usia dan memberikan hasil yang keliru. Oleh karena itu, permasalahan ini diatasi dengan melakukan *feature scaling*.

8. Persamaan berikut digunakan untuk menghitung standarisasi pada *feature scaling*:



Gambar 4. Persamaan Standarisasi untuk *feature scaling*

Implementasi standarisasi pada studi kasus ini menggunakan ***StandardScaler*** yang dapat dituliskan dalam instruksi berikut:

```
#StandardScaler untuk feature scaling
from sklearn.preprocessing import StandardScaler

st_x= StandardScaler()
X_train[:, 1:]= st_x.fit_transform(X_train[:, 1:])
X_test[:, 1:]= st_x.transform(X_test[:, 1:])
```

Catatan:

- Class ***sklearn.preprocessing.StandardScaler*** berfungsi untuk standarisasi fitur dengan menghilangkan *mean* dan menskalakan ke unit varians. Informasi lebih detil terkait *class* ini dapat dipelajari di website berikut: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.
- **[:, 1:]** artinya mengambil semua baris (:) dari array dan semua kolom dari indeks ke-1.
- Amati variabel ***X_train*** dan ***X_test*** setelah dilakukan standarisasi! Bandingkan dengan nilai kedua variabel tersebut sebelum distandarisasi.

Latihan:

Standarisasi variabel ***X_train*** dan ***X_test*** menggunakan class ***sklearn.preprocessing.MinMaxScaler***!

KODE PROGRAM GABUNGAN

Berikut ini merupakan kode program gabungan, mulai dari import pustaka yang dibutuhkan untuk menyelesaikan studi kasus hingga operasi *feature scaling*.

```
#import libraries
import numpy as np
import pandas as pd
import sklearn

#import dataset
dataset=pd.read_csv('Data.csv')
df=pd.DataFrame(dataset)

#ekstract independent variables
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

#identifikasi missing values
print(df.info())
mv=df.isna().sum()
print('\nJumlah missing value tiap kolom:\n', mv)

'''
Solusi 1: Hapus data yang memiliki missing value
'''
df1=df.copy()

#ukuran data awal
print("Sebelum: ", df1.shape)

#hapus baris yang memiliki missing value
df1.dropna(inplace=True)

#ukuran data akhir
print("Setelah: ", df1.shape)

'''
Solusi 2: Isi missing value dengan value lain
'''
df2 = df.copy()

#isi missing value dengan nilai kolom rata-rata (mean)
df2['Age'].fillna(df['Age'].mean(), inplace=True)
df2['Salary'].fillna(df['Salary'].mean(), inplace=True)

'''
Isi missing value pada variabel X
'''
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
```

```
imputer.fit(X[:, 1:3])
X[:, 1:3] = imputer.transform(X[:, 1:3])

#pengodean kolom Country dengan LabelEncoder
from sklearn.preprocessing import LabelEncoder

label_encoder_x= LabelEncoder()
X[:, 0]= label_encoder_x.fit_transform(X[:, 0])

#pengodean kolom Purchased dengan LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)

#pembagian dataset menjadi training dan test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)

#StandardScaler untuk feature scaling
from sklearn.preprocessing import StandardScaler

st_x= StandardScaler()
X_train[:, 1:]= st_x.fit_transform(X_train[:, 1:])
X_test[:, 1:]= st_x.transform(X_test[:, 1:])
```

Laporan Praktikum:

Laporan praktikum yang berisi modifikasi kode program yang telah diubah berdasarkan latihan praktikum diunggah melalui form: https://forms.gle/znCaFz4R71FDKVfX7 paling lambat H-1 sebelum perkuliahan minggu depan. Kolom "Tugas Pertemuan" pilih pertemuan "2."

Sumber Belajar Tambahan:

Sumber belajar tambahan dengan studi kasus untuk memperdalam pemahaman penanganan *Missing Data*, *Categorical Data*, *Data Splitting*, dan *Feature Scaling* dapat diakses pada website: https://dotnettutorials.net/lesson/data-preprocessing-in-data-science/ yang datasetnya dapat diakses pada website berikut ini: https://www.kaggle.com/c/titanic/data.